

NAAN MUDHALVAN PROJECT
JSON Refiner Advanced Edition

K.R.P College of Arts and Science
(Affiliated to Madurai Kamaraj University)
kakkivadanpatti, sivakasi - 626124



Submitted by

A.Sathya (C3S44964)

S.Avrin Beevi (C3S44951)

M.Madhumitha (C3S44957)

G.Paripoorani (C3S44961)

T.Ariharasudhan (C3S44966)

Abstract

JSON Refiner Advanced Edition is a professional tool designed to process, validate, and transform JSON data efficiently. Modern applications frequently exchange data in JSON format, but inconsistencies in data structure, naming conventions, and data types often create difficulties for developers. The proposed system solves this issue by providing automated JSON processing features such as parsing, validation, case conversion, flattening, and merging. The system is implemented using Python and provides an interactive interface using the Gradio framework. This allows users to refine and transform JSON data without requiring complex programming knowledge. The project demonstrates how modern tools and libraries can be combined to build an intelligent JSON processing platform.

Introduction

JSON (JavaScript Object Notation) is one of the most commonly used formats for data exchange between web services, APIs, and applications. Although JSON is lightweight and easy to read, large JSON datasets often become difficult to manage. Developers must frequently validate data, convert key naming conventions, and restructure nested objects. Manual processing of JSON files can lead to errors and consume significant development time.

JSON Refiner Advanced Edition was developed to simplify these tasks by automating common JSON processing operations. The system accepts user input data, applies intelligent transformations, and produces structured JSON output ready for use in applications.

Problem Statement

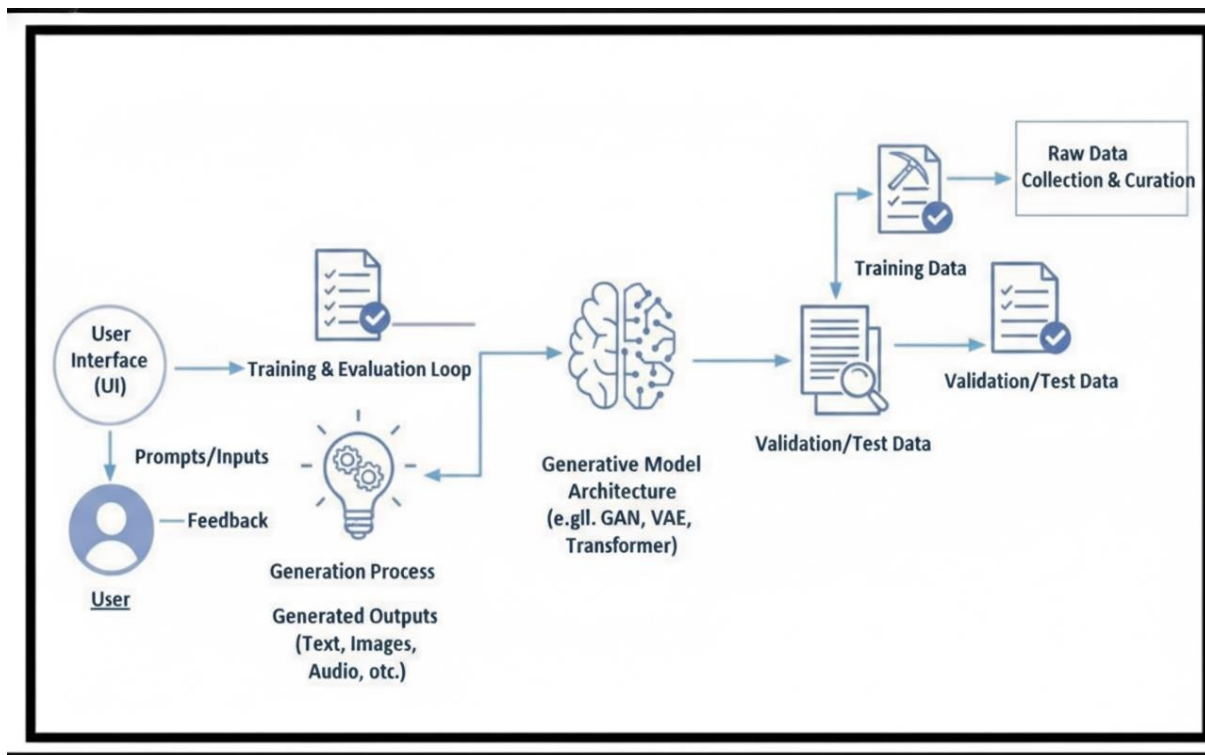
Organizations frequently receive JSON data from multiple external sources. These sources often follow different formatting rules and naming conventions. As a result, developers must spend time cleaning and restructuring JSON files before using them in systems. Common problems include inconsistent key naming, incorrect data types, deeply nested structures, and missing fields. Without automation, resolving these problems manually can slow down development and increase the risk of errors. The project aims to provide an automated system to refine JSON data efficiently.

Objectives

The main objective of this project is to develop an efficient tool that simplifies JSON data processing. The system aims to automatically detect and correct JSON data types, validate the structure using JSON schema, and transform JSON data into a standardized format. Another objective is to provide different naming convention options to maintain consistency in data structures. The project also focuses on building a user-friendly interface that allows users to easily upload, process, and refine JSON data without requiring complex programming knowledge.

System Architecture

The system architecture consists of several modules working together. The input module collects raw data from users. The processing engine converts raw key-value data into structured JSON objects. The validation module checks the correctness of JSON structures using schema validation rules. The transformation module applies case conversions, flattening, and merging operations. Finally, the output module displays the refined JSON data through the Gradio interface.



Technologies Used

Python is used as the core programming language due to its simplicity and extensive library support. JSON5 is used for flexible JSON parsing. JSONSchema library is used to validate JSON structures. Regular expressions are used for text pattern matching and case conversions. Gradio is used to create an interactive web interface that allows users to enter data and view results instantly. Google Colab can be used as the development and testing environment.

Modules Description

The system is divided into multiple modules to perform different tasks in JSON processing. The JSON parsing module reads and analyzes the input JSON data. The schema validation module checks whether the JSON structure follows the required schema rules. The data type detection module automatically identifies and corrects the data types present in the JSON file. The transformation module refines the JSON data by applying naming conventions and formatting rules. Finally, the user interface module allows users to upload, process, and download refined JSON data easily.

Testing and Validation

Testing and validation are important steps to ensure the reliability of the system. Different types of JSON data are tested to check whether the system can correctly parse, validate, and refine them. Schema validation is used to verify that the JSON data follows the required structure. Error handling mechanisms are also tested to ensure the system can detect and report issues in the input data. Through proper testing, the system ensures accurate results and stable performance.

Advantages

JSON Refiner Advanced Edition offers several advantages for users working with JSON data. It reduces the time required to clean and format JSON files. The system automatically detects data types and validates JSON structures, reducing the chances of errors. It also provides consistent naming conventions and organized output. The user-friendly interface makes the tool easy to use even for beginners, making it useful for developers, analysts, and students.

Sampling Python Implementation

```
import json
import gradio as gr

def refine_json(text):
    data = {}
    lines =
    text.split("\n") for
    line in lines:
        if ":" in line:
            k,v = line.split(":",1)
            data[k.strip()] = v.strip()
    return json.dumps(data,
    indent=4)

interface =
    gr.Interface( fn=refi
    ne_json,
    inputs="textbox",
    outputs="textbox",
    title="JSON Refiner Advanced Edition"
)

interface.launch()
```

```
import json
import gradio as gr

def refine_json(text):
    data = {}
    lines =
    text.split("\n") for
    line in lines:
        if ":" in line:
            k,v = line.split(":",1)
```

```
        data[k.strip()] = v.strip()
    return json.dumps(data,
        indent=4)
```

```
interface =
    gr.Interface( fn=refi
        ne_json,
        inputs="textbox",
        outputs="textbox",
        title="JSON Refiner Advanced Edition"
    )
```

```
interface.launch()
```

```
import json
```

```
import gradio as gr
```

```
def refine_json(text):
```

```
    data = {}
```

```
    lines =
```

```
    text.split("\n") for
```

```
    line in lines:
```

```
        if ":" in line:
```

```
            k,v = line.split(":",1)
```

```
            data[k.strip()] = v.strip()
```

```
    return json.dumps(data,
```

```
    indent=4)
```

```
interface =
```

```
    gr.Interface( fn=refi
```

```
    ne_json,
```

```
    inputs="textbox",
```

```
    outputs="textbox",
```

```
    title="JSON Refiner Advanced Edition"
```

```
)
```

```
interface.launch()
```

```
import json
```

```
import gradio as gr
```

```
def refine_json(text):
```

```
    data = {}
```

```

lines =
text.split("\n") for
line in lines:
    if ":" in line:
        k,v = line.split(":",1)
        data[k.strip()] = v.strip()
return json.dumps(data,
indent=4)

    interface = gr.Interface(
fn=refine_json,
inputs="textbox",
outputs="textbox",
title="JSON Refiner Advanced Edition"
)

interface.launch()

```

```

import json
import gradio as gr

def refine_json(text):
    data = {}
    lines =
text.split("\n") for
line in lines:
    if ":" in line:
        k,v = line.split(":",1)
        data[k.strip()] = v.strip()
    return json.dumps(data,
indent=4)

interface =
gr.Interface( fn=refi
ne_json,
inputs="textbox",

```

```
    outputs="textbox",  
    title="JSON Refiner Advanced Edition"  
)
```

```
interface.launch()
```

```
import json  
import gradio as
```

```
gr def
```

```
    refine_json(text)
```

```
    data = {}
```

```
        lines =  
        text.split("\n")  
        for  
        line in lines:  
            if ":" in line:  
                k,v = line.split(":",1)  
                data[k.strip()] = v.strip()  
        return json.dumps(data,  
        indent=4)
```

```
interface =  
    gr.Interface( fn=refi  
    ne_json,  
    inputs="textbox",  
    outputs="textbox",  
    title="JSON Refiner Advanced Edition"  
)
```

```
interface.launch()
```

```

import json
import gradio as gr

def refine_json(text):
    data = {}
    lines =
    text.split("\n") for
    line in lines:
        if ":" in line:
            k,v = line.split(":",1)
            data[k.strip()] = v.strip()
    return json.dumps(data,
    indent=4)

interface =
    gr.Interface( fn=refi
    ne_json,
    inputs="textbox",
    outputs="textbox",
    title="JSON Refiner Advanced Edition"
)

interface.launch()
import json
import gradio as gr

def refine_json(text):
    data = {}
    lines =
    text.split("\n") for
    line in lines:
        if ":" in line:
            k,v = line.split(":",1)
            data[k.strip()] = v.strip()
    return json.dumps(data,
    indent=4)

interface =
    gr.Interface( fn=refi

```



```
ne_json,  
inputs="textbox",  
outputs="textbox",  
title="JSON Refiner Advanced Edition"  
)
```

```
interface.launch()
```

```
import json  
import gradio as gr
```

```
def refine_json(text):  
    data = {}  
    lines =  
    text.split("\n")  
    for line in lines:  
        if ":" in line:  
            k,v = line.split(":",1)  
            data[k.strip()] = v.strip()  
    return json.dumps(data,  
        indent=4)
```

```
interface = gr.Interface(  
    fn=refine_json, inputs="textbox", outputs="textbox",  
    title="JSON Refiner Advanced Edition"  
)
```

```
interface.launch()
```

```
import json  
import gradio as gr
```

```
def refine_json(text):
    data = {}
    lines =
text.split("\n") for
line in lines:
    if ":" in line:
        k,v = line.split(":",1)
        data[k.strip()] = v.strip()
    return json.dumps(data,
indent=4)

interface =
gr.Interface( fn=refi
ne_json,
inputs="textbox",
outputs="textbox",
title="JSON Refiner Advanced Edition"
)

interface.launch()
```

```
import json
import gradio as gr

def
refine_json(text
): data = {}
lines = text.split("\n")
for line in lines:
    if ":" in line:
        k,v = line.split(":",1)
        data[k.strip()] = v.strip()
    return json.dumps(data,
indent=4)
```

```
interface =  
    gr.Interface( fn=refi  
ne_json,  
    inputs="textbox",  
    outputs="textbox",  
    title="JSON Refiner Advanced Edition"  
)
```

```
interface.launch()
```

```
import json  
import gradio as gr
```

```
def refine_json(text):  
    data = {}  
    lines =  
text.split("\n")  
    for  
line in lines:  
        if ":" in line:  
            k,v = line.split(":",1)  
            data[k.strip()] = v.strip()  
    return json.dumps(data,  
indent=4)
```

```
interface =  
    gr.Interface( fn=refi  
ne_json,  
    inputs="textbox",  
    outputs="textbox",  
    title="JSON Refiner Advanced Edition"  
)
```

```
interface.launch()
```

Future Enhancement

In the future, the system can be enhanced by adding more advanced features such as support for multiple data formats like XML and CSV. Integration with cloud platforms can allow users to process JSON data online. Additional features like real-time collaboration, advanced error detection, and automated data visualization can also improve the functionality of the system. These improvements will make the tool more powerful and suitable for large-scale data processing.

Conclusion

SON Refiner Advanced Edition is an efficient tool designed to simplify JSON data processing. By providing features such as parsing, validation, data type detection, and transformation, the system helps users refine and organize JSON data effectively. The use of Python and Gradio makes the system both powerful and user-friendly. Overall, the project demonstrates how automated tools can improve data accuracy, reduce manual effort, and enhance productivity in modern software development.